

TEKNOFEST

HAVACILIK, UZAY VE TEKNOLOJİ FESTİVALİ

ULAŞIMDA YAPAY ZEKA YARIŞMASI

KRİTİK TASARIM RAPORU

TAKIM ADI: Bingöl STEM RoboKulüp - UYZ

BAŞVURU ID: 363992

İÇİNDEKİLER

İÇİNDEKİLER	2
ŞEKİL LİSTESİ	3
TABLO LİSTESİ	HATA! YER İŞARETİ TANIMLANMAMIŞ.
KISALTMALAR	5
1. TAKIM ŞEMASI	6
2. PROJE MEVCUT DURUM DEĞERLENDİRMESİ	6
3.ALGORİTMALAR VE SİSTEM MİMARİSİ	7
3.1 Veri Setleri	7
3.1.1	7
3.1.2	7
3.1.3	7
3.2.Algoritmalar	7
3.2.1 CenterNet HourGlass104	7
3.2.2 Sistem Mimarisi	8
4.ÖZGÜNLÜK	9
4.1 UAP ve UAI'lerin Tespiti	9
4.2. Pipeline.config Ayarları	9
5.SONUÇLAR VE İNCELEME	10
6.KAYNAKÇA	11



ŞEKİL LİSTESİ

Şekil 1 Takım Şeması	6
Şekil 2 CenterNet karşılaştırmalı AP Metriği Eğirisi (solda). CenterNet Modeli Şematik Canlandırılması ve Nesne Takibine uygun yapısının gösterilmesi (Sağda)	7
Şekil 3 Sistemin Eğitim ve Test Algoritması	8
Şekil 4 TensorBoard eğitim-kayıp eğrileri	10



TABLO LİSTESİ

Tablo 1 Takım Görev Dağılım Tablosu	6
Tablo 2 CenterNet Hız ve COCO mAP metriği değerleri	8

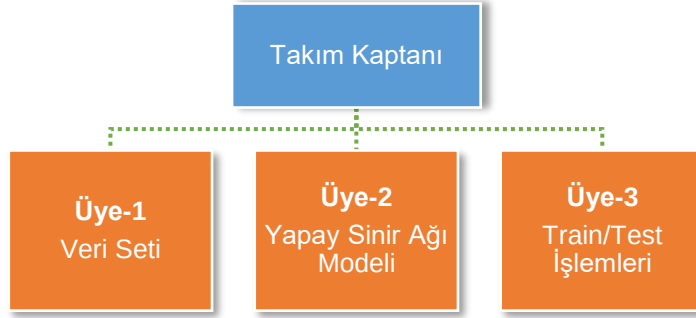


KISALTMALAR

Bkz.	bakınız
NMS	Non Maximum Suppression
TF	TensorFlow
API	Application Programming Interface
JSON	JavaScript Object Notation
CSV	Comma Seperated Values



1. TAKIM ŞEMASI



Şekil 1

Danışman	Görev dağılımını yapar. Başvuru sürecini ve sonrasındaki yarışma sürecinde takımın danışmanlığını yapar. Takımın iletişim sorumlusudur. Tüm çalışma ve denemelerde takıma eşlik eder.
Takım Kaptanı	Takım kaptanı olarak Yazılımın tüm yapım ve algoritma süreçlerinden sorumludur. Özellikle ilk algoritma geliştirme ve yazım aşamasına liderlik eder. Yarışma şartnamesine göre donanımsal problemlerin çözümü ve algoritmaların tasarımı işlemlerini yürütür.
Üye-1	İnternet ortamından Açık Kaynak Veri Setlerini araştırır, eldeki Veri Setlerini ile ilgili tüm çalışmalardan sorumludur.
Üye-2	Kullanılacak Nesne Tespit Sinir Ağlarını araştırıp ilgili literatür taramaları ve metriklerini inceler ve takıma sunar.
Üye-3	Takımın hazırladığı algoritma ve veri setlerine uygun şekilde Google COLAB ve Laptop ile Train ve Test süreçlerini yürütür.

Tablo 1 Takım Görev Dağılım Tablosu

2. PROJE MEVCUT DURUM DEĞERLENDİRMESİ

Ön Tasarım Raporunda kullanılmasını planladığımız TensorFlow 2 Detection Model Zoo koleksiyonundan CenterNet_HourGlass104 ve Faster-R-CNN_Resnet_101 dedektörlerini veri setlerimiz ile test ettik. Testler sırasında dikkatimizi çeken bulgular FasterRCNN_Resnet_101 dedektörünün CenterNet_HourGlass104 dedektörüne kıyasla çok daha hızlı çalıştığı fakat küçük nesneleri (özellikle insan ve motosiklet) veya irtifaya göre küçük kalan nesneleri algılamada göre nisbeten başarısız olduğuydu. Bu hız avantajını değerlendirme adına model hiper parametreleri ayarlandı, artırma (augmentation) çeşitliliği artırıldı hatta overfit olmaması için dropout yapılarak eğitim adımı (train step) sayısı artırılarak train loss'un daha düşük değerlere inmesi için modelin eğitilmesi süreci uzatıldı veri seti artırıldı fakat yine CenterNet_HourGlass104 ağının performansını vermediği için CenterNet_HourGlass104 modelinin kullanılmasına karar verildi. Yaptığımız deneyler sonucunda var olan test donanımlarımız ile CenterNet_HourGlass104 ağına ait sonuçların daha iyi olduğu gözlemlendi. Yarışma süre sınırlamalarında herhangi bir değişiklik yaşanmazsa CenterNet_HourGlass104 ağı ile eğitim ve test işlemlerine devam edilecektir.

3.ALGORİTMALAR VE SİSTEM MİMARİSİ

3.1 Veri Setleri

3.1.1.

Veriseti oluştururken okulumuzda bulunan DJI Mavic Pro 2 drone'umuz ile şartnameye uygun açıda ve yüksekliklerde çekimler yaptık. Bu videolardan yarışmadaki frame oranına göre resim alacak bir python betiği yazarak ayrılıp etiketleme yapıldı. Yarışma formatına en uygun görüntüler bu veri seti ile sağlandığı ve istediğimiz çözünürlük ve bu nedenle kullanılan en verimli veriseti oldu. İkinci ve akşam saatleri arası ve karlı çekimler konusunda da ayrıca efekt uygulamadan orijinal çekimler yapmamızı sağlamıştır.

3.1.2.

Visdrone verisetinden uygun yükseklik ve açıdan elde edilen fotoğraflar veriseti kümemize eklendi. Sağladığı insan ve taşıt çeşitliliğinin fazla olmasından dolayı veri setimizin verimliliğini arttırdı [1].

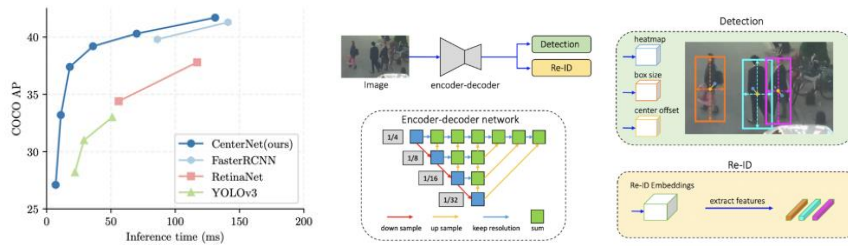
3.1.3.

Stanford üniversitesinin yayınladığı Stanford drone verisetini kümemize ekledik. Bu veriseti büyüklüğüyle test sonuçlarını doğrudan etkilemiştir [2] .

3.2.Algoritmalar

3.2.1 CenterNet HourGlass104

TensorFlow-2 Model Zoo koleksiyonunda ücretsiz olarak sunulan ön eğitilmiş nesne tespit ağıdır. Yaptığımız araştırmalarda ve ayrıca veriseti üzerindeki testlerimizde özellikle küçük insan görüntülerini oldukça iyi tespit ettiğini deneyimledik. Ağı yapısını incelediğimizde, CenterNet ağı adından da anlaşılacağı üzere nesneleri tespit ederken sınırlayıcı kutuların merkez noktaları olarak modeller ve merkez noktalarını bulmak için temel nokta tahminini kullanır ve boyut, 3B konum, yön ve hatta poz gibi diğer tüm nesne özelliklerine geri döner [3]. Yön özelliği Çoklu Nesne Takibi (MOT) avantajı sağlar ki bu avantaj video içerisindeki nesneleri framerlerin geçişi sırasında takip imkanı sağlayarak problemimize yönelik nesne tespiti yönünü güçlendirir (bkz. Şekil 2). Merkez noktaya odaklanma özelliği ağı kendine has bazı avantajlar sunar bunlar; Nesne sınırlandırıcı kutusunun merkezine odaklanacağı için IoU konusunda avantajlı olacağını değerlendirmekteyiz. Aynı avantajı YOLO ve SSD gibi çapa tabanlı dedektörlerde karşılaşılan aynı nesneyi birden fazla tespit etme ve bunu aşmak için kullanılan Non Max Supression (NMS) algoritmasına ihtiyaç duymaz ve ek bir algoritma ihtiyacı ortadan kalkacağı için bu da hesaplama maliyetini düşürmektedir [4]. Backbone olarak HourGlass104'ü kullanır. Hava görüntülerinden tespit için verimli sonuçlar vermiştir. Yukarıda bahsedilen tüm avantajlı ve güçlü özellikleri bizim bu ağı kullanmamızda karar verme nedeni olmuştur [5].



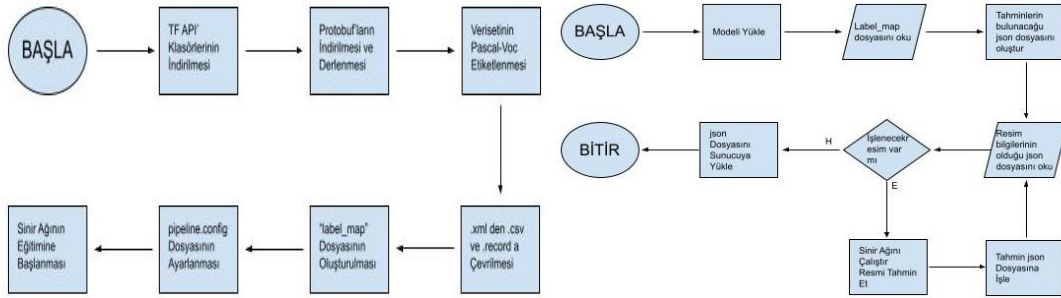
Şekil 2 CenterNet karşılaştırmalı AP Metriği Eğirisi (solda). CenterNet Modeli Şematik Canlandırılması ve Nesne Takibine uygun yapısının gösterilmesi (Sağda)

Model	Hız(ms)	COCO mAP
CenterNet HourGlass104 512x512	70	41.9
CenterNet HourGlass104 1024x1024	197	44.5

Tablo 2 CenterNet Hız ve COCO mAP metriği değerleri

3.2.2 Sistem Mimarisi

Programlama dili olarak Python'ı Anaconda-Spyder ve Anaconda-Jupyter ortamlarında kullanacağız. Yapay zekâ kütüphanesi olarak; train işlemleri için Tensorflow-1.14-gpu ve genel kullanım ve test işlemleri için daha hızlı olduğundan dolayı Tensorflow 2.2.0'ı kullanacağız.



Şekil 3 Sistemin Eğitim ve Test Algoritması

Sinir ağıımızın train işlemini gerçekleştirirken aşağıdaki adımları izlememiz gerekmektedir:

- Tensorflow Object Detection API kurulumunu gerçekleştirirken kurumsal github deposundan belirli bir ağaç yapısı ile klonlamamız gerekmektedir.
- TF API model ve eğitim parametrelerini yapılandırabilmek için protobufların indirilip derlenmesi gerekmektedir.
- Veriseti'ni Pascal-VOC formatında etiketledik. Etiketleme için labelimg uygulamasının kurulması gerekmektedir. Daha sonra sınıfların ve ID'lerinin bulunduğu label_map.pbtxt dosyası oluşturulmalıdır.
- ".xml" uzantılı dosyaların ".record" uzantılı dosyalara çevrilmesi için API'de bulunan "xml_to_csv.py" betiği ile önce ".csv"ye sonrada "generate_record.py" betiği ile de ".record" dosyalarına çevrilmelidir.
- Son olarak her ağın yanında gelen "pipeline.config" dosyasındaki ayarlar sinir ağına göre yapılmalıdır.
- Bu adımlardan sonra eğitim işlemine geçilebilir.



Fotoğraf 1 Model test çıktıları

4.ÖZGÜNLÜK

4.1 UAP ve UAI'lerin Tespiti

UAP ve UAI'lerin iniş durumu belirlenirken iniş alanının tamamının resim içinde olup olmadığını tespit etmek amacıyla sınır ağından gelen UAP veya UAI tahmini geçici bir resim olarak kaydedilecektir. Daha sonra kaydedilen resimde "OpenCV" kütüphanesi kullanılarak bütün bir dairenin olup olmadığına bakılacaktır. Dairenin bulunması durumunda inişe uygun aksi takdirde inişe uygun değil değerleri JSON dosyasına işlenecektir.



Fotoğraf 2 UAP ve UAI veri setinden bir görüntü

4.2 Pipeline.config Ayarları

Train işlemi sırasında ağıla ilgili ayarların yapıldığı "pipeline.config" dosyasında "data_augmentation_options"lar eklendi. Ağı aşırı öğrenme(overfit) olmaması için "use_dropout:true" parametresi pipeline dosyasına eklendi. Train adımları ilerledikçe "learning_rate"i azaltmak amacıyla schedule{} parametresi pipeline dosyasına eklendi.

4.3 Veri Arttırma(Data Augmentation)

Hava koşullarındaki değişikliklerin sorun teşkil etmemesi için eğitim veri setine veri arttırma parametreleri aşağıdaki parametreler verilmiştir.

- RandomHorizontalFlip random_horizontal_flip
- RandomAdjustBrightness random_adjust_brightness
- RandomAdjustContrast random_adjust_contrast
- RandomAdjustHue random_adjust_hue
- RandomAdjustSaturation random_adjust_saturation
- RandomDistortColor random_distort_color

4.4 Taşıtların Sınıflandırılması

Taşıtların, özellikle motosiklet ve kamyonet gibi taşıtların birbirine benzememesinin modelimizi zorlayacağından dolayı taşıtlar kendi arasında: iş makinesi, kamyonet, araba gibi sınıflara ayrılmıştır.

5.SONUÇLAR VE İNCELEME

Ön tasarım raporunda da belirttiğimiz gibi CenterNet HourGlass104 ağı Tensorflow 2.2.0 da Google colab ortamında train edildi. Faster-R-CNN-ResNet101 ağı da Tensorflow 1.14 kütüphanesi ile Google colab ortamında train edildi. Her 2 sinir ağında da taşıtların bulunmasında insanların bulunmasına göre daha başarılı olduğu saptandı. Bunun nedeni veriseti kümемizde taşıt sayısının insan sayısına göre daha fazla olmasıdır. Verisetini dengelemek amacıyla Visdrone verisetinin 5. Görevinde verilen crowd counting challenge resimlerini de sinir ağlarımızı eğitirken kullanacağız.

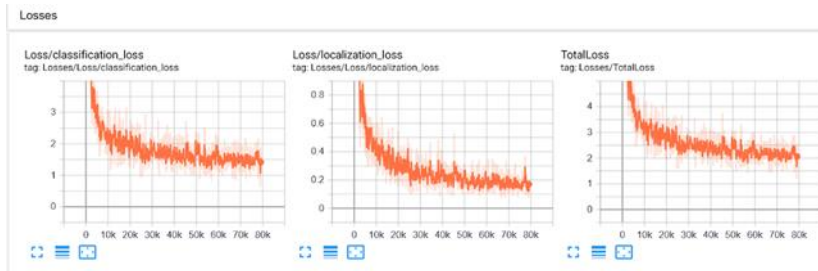
Centernet sinir ağı ile anaconda-prompt ekranında test işlemi gerçekleştirirken her bir resim için tahmin süresinin 0.7 saniye ile 1 saniye arasında değiştiğini saptadık. Faster-R-CNN sinir ağında ise bir resim için tahmin süresinin 0.8 saniyenin altında olduğunu gözlemledik. Yarışma tarihine kadar süre sınırlanmalarında bir değişiklik olmazsa Centernet ağını kullanmayı planlamaktayız. Süre sınırlamalarında bir değişiklik olursa Faster-R-CNN-ResNet101 sinir ağını kullanabiliriz.

```
Anaconda Prompt
Running inference for data/models/foto/testSet1/2_frame5516.jpg... sure= 0.7588808141174316
Running inference for data/models/foto/testSet1/2_frame5520.jpg... sure= 0.7512724700164995
Running inference for data/models/foto/testSet1/2_frame5524.jpg... sure= 0.74853801779541
Running inference for data/models/foto/testSet1/2_frame5528.jpg... sure= 0.880444051452637
Running inference for data/models/foto/testSet1/2_frame5532.jpg... sure= 0.7550804215020752
Running inference for data/models/foto/testSet1/2_frame5536.jpg... sure= 0.756860024261495
Running inference for data/models/foto/testSet1/2_frame5540.jpg... sure= 0.7489354610443115
Running inference for data/models/foto/testSet1/2_frame5544.jpg... sure= 0.7544336318969727
Running inference for data/models/foto/testSet1/2_frame5548.jpg... sure= 0.757972420311047
Running inference for data/models/foto/testSet1/2_frame5552.jpg... sure= 0.7530289213012227
Running inference for data/models/foto/testSet1/2_frame5556.jpg... sure= 0.7512784804211426
Running inference for data/models/foto/testSet1/2_frame5560.jpg... sure= 0.7547471522284912
Running inference for data/models/foto/testSet1/2_frame5564.jpg... sure= 0.7524050061846143
Running inference for data/models/foto/testSet1/2_frame5568.jpg... sure= 0.7569746971130371
Running inference for data/models/foto/testSet1/2_frame5572.jpg... sure= 0.75337934640401855
Running inference for data/models/foto/testSet1/2_frame5576.jpg... sure= 0.8488023936462802
Running inference for data/models/foto/testSet1/2_frame5580.jpg... sure= 0.75513863563576
Running inference for data/models/foto/testSet1/2_frame5584.jpg... sure= 0.7539782524188807
Running inference for data/models/foto/testSet1/2_frame5588.jpg... sure= 0.7574689502713212
Running inference for data/models/foto/testSet1/2_frame5592.jpg... sure= 0.754434612719336
Running inference for data/models/foto/testSet1/2_frame5596.jpg... sure= 0.7599684728851318
Running inference for data/models/foto/testSet1/2_frame5600.jpg... sure= 0.7483801649910889
Running inference for data/models/foto/testSet1/2_frame5604.jpg... sure= 0.7507474422454834
Running inference for data/models/foto/testSet1/2_frame5608.jpg... sure= 0.7579982280731201
Running inference for data/models/foto/testSet1/2_frame5612.jpg... sure= 0.757993465380249
Running inference for data/models/foto/testSet1/2_frame5616.jpg... sure= 0.7565724849700928
Running inference for data/models/foto/testSet1/2_frame5620.jpg... sure= 0.7574148178180586
Running inference for data/models/foto/testSet1/foto_10080.jpg... sure= 0.9257278442302132
Running inference for data/models/foto/testSet1/foto_10010.jpg... sure= 0.895782000656279
Running inference for data/models/foto/testSet1/foto_10012.jpg... sure= 0.9363763332366943
Running inference for data/models/foto/testSet1/foto_10044.jpg... sure= 0.9215819835662842
Running inference for data/models/foto/testSet1/foto_10064.jpg... sure= 0.92029296479
Running inference for data/models/foto/testSet1/foto_10080.jpg... sure= 0.948741300582857
Running inference for data/models/foto/testSet1/foto_10096.jpg... sure= 0.9446234703063965
Running inference for data/models/foto/testSet1/foto_10112.jpg... sure= 0.945520711808037
Running inference for data/models/foto/testSet1/foto_10128.jpg... sure= 0.965013511889648
Running inference for data/models/foto/testSet1/foto_10144.jpg... sure= 0.9383407669067383
Running inference for data/models/foto/testSet1/foto_10160.jpg... sure= 0.9201814513401211
Running inference for data/models/foto/testSet1/foto_10176.jpg... sure= 0.941401575088501
Running inference for data/models/foto/testSet1/foto_10192.jpg... sure= 0.9449357986450195
Running inference for data/models/foto/testSet1/foto_10208.jpg... sure= 0.931487723809877
Running inference for data/models/foto/testSet1/foto_10224.jpg... sure= 0.93147802152969217
Running inference for data/models/foto/testSet1/foto_10240.jpg... sure= 0.959916851864624
Running inference for data/models/foto/testSet1/foto_10256.jpg... sure= 0.9248415497283936
Running inference for data/models/foto/testSet1/foto_10272.jpg... sure= 0.922671858673996
Running inference for data/models/foto/testSet1/foto_10288.jpg... sure= 0.958706958770752
Running inference for data/models/foto/testSet1/foto_10304.jpg... sure= 0.92047257592041
Running inference for data/models/foto/testSet1/foto_10320.jpg... sure= 0.942896364191848
Running inference for data/models/foto/testSet1/foto_10336.jpg... sure= 0.9393093585968018
```

Fotoğraf 3 CenterNET_HourGlass104 modeli eğitim adımları ekran görüntüsü



Fotoğraf 4 Modelimize ait bazı test çıktıları



Şekil 4 TensorBoard eğitim-kayıp eğrileri

6.KAYNAKÇA

- [1] «Github,» 18 Ekim 2021. [Çevrimiçi]. Available: <https://github.com/VisDrone/VisDrone-Dataset>. [Erişildi: 1 Ocak 2022].
- [2] «Stanford University,» 8 Ocak 2016. [Çevrimiçi]. Available: https://cvgl.stanford.edu/projects/uav_data/. [Erişildi: 12 Ocak 2022].
- [3] «gaunghan,» 6 Nisan 2020. [Çevrimiçi]. Available: <http://gaunghan.info/blog/en/my-thoughts/centernet-and-its-variants/>. [Erişildi: 11 Haziran 2022].
- [4] «victordibia,» 21 Mart 2021. [Çevrimiçi]. Available: <https://victordibia.com/blog/state-of-object-detection/>. [Erişildi: 29 Temmuz 2022].
- [5] V. K. S. S. C. Dheeraj Reddy Pailla, «Object detection on aerial imagery using CenterNet,» *arXiv*, pp. 1-2, 22 Ağustos 2019.

